



Trabalho Individual

Matéria: **Programação Web**

Turma: **2026/1**

Professor: **Douglas Tagliari**

Nome Completo

RA

Data

Nota

Objetivo

Avaliar, de forma prática, todo o conteúdo da ementa da disciplina por meio de código funcional, documentação e apresentação.

O objetivo é demonstrar domínio dos princípios fundamentais da programação web, abrangendo estrutura de código, protocolos de comunicação, boas práticas e padrões de desenvolvimento.

Também busca verificar a compreensão dos princípios de design de páginas web, considerando acessibilidade e responsividade, bem como o entendimento de projetos e arquiteturas de software aplicados ao Front-End e Back-End.

Entrega e Apresentação

A entrega deverá ser feita via GitHub, contendo o front-end, as APIs de back-end, um README completo, mocks/dados e documentação simples (Swagger/OpenAPI).

O projeto deve estar plenamente funcional e será apresentado à turma, com tempo máximo de 15 minutos por aluno.

A apresentação terá roteiro livre, mas deve contemplar: arquitetura do sistema, demonstração do uso, funcionamento das APIs e análise técnica utilizando DevTools e Lighthouse.

Critérios de avaliação (20% da Nota Final)

1. Atendimento ao Escopo (0,2pts)
2. Funcionalidade e Estabilidade (0,2pts)
3. APIs REST Funcionais (0,2pts)
4. Avaliação Técnica Código (0,2pts)
5. Apresentação e Clareza (0,2pts)

Bônus (até +0,2 pts extras)

Tem como objetivo valorizar a aplicação de práticas avançadas de desenvolvimento web.

O **Trabalho Individual representa 20%** da nota final da disciplina. O desempenho de bônus neste trabalho poderá gerar acréscimo direto **na nota global da disciplina**. O acréscimo será limitado a dois pontos percentuais na nota global, se atingir +0,2 pts extras no trabalho individual.

Exemplo: Aluno tem nota final 8,3 e conquistou 0,2 pts de bônus, terá acréscimo de +2 pontos percentuais na nota final, que será ajustada para 8,5.

Critérios para avaliação e obtenção deste bônus serão descritos como requisitos extras no material, sessão 5.

1) Objetivo

Implementar uma simulação de **e-commerce** com uma estrutura básica de front-end e **APIs**, evidenciando domínio de **modelo cliente-servidor**, **HTTP/REST**, **tratamento de erros**, **persistência simples de dados** e **código organizado**.

O aluno deverá selecionar um tema de sua preferência para desenvolver o e-commerce. Será permitido o uso de Inteligência Artificial, desde que o estudante seja capaz de explicar as estruturas e construções realizadas por meio dela.

2) Requisitos de Front-end

Páginas obrigatórias

- **Home:** (/)
 - Pelo menos um Banner, Menu de navegação e barra de pesquisa, exibição de 4 produtos.
- **Busca e Listagem de Produtos:** (/busca?query=blusa)
 - Busca de produtos, pesquisando através do nome e seus atributos.
- **Página de Detalhes de Produto (PDP):** (/p/nome/:id)
 - Exibir os dados completos do produto e ação "Adicionar ao carrinho".
- **Carrinho** (/carrinho)
 - Apresentar itens, quantidade, subtotal, frete simulado, cupom e total

Comportamento e Técnica

- **Seleção de tecnologia:** pode ser utilizada solução em NodeJS (base de estudos e exercícios em aula) ou frameworks/libs à escolha, desde que **demonstre domínio e compreensão sobre conceitos e a base**.
- **Integração com APIs**, validação e tratamento de erros.
- **Carrinho:** Exibir os produtos adicionados através de **LocalStorage** e carregando informações atualizadas do banco de dados.
- **UI/UX:** Construção de uma estrutura moderna com uma excelente experiência de usuário.
- **Logs de debug mínimos** no console quando apropriado; Evitar dados sensíveis.

3) Requisitos de Back-end

Implementar as seguintes **rotas de APIs REST** .

- **GET /health**
 - Validação de funcionamento do serviço
- **POST /products** (rota autenticada com basic auth) (cadastro dos produtos)
 - Ao criar um produto, incluir atributos (ex. Cor, Tamanho, Peso, Descrição)
 - Deverá funcionar utilizando o Postman
- **DELETE /product/:id** (rota autenticada com basic auth) (deleção de produto)
 - Deletar um produto específico passando o ID
 - Deverá funcionar utilizando o Postman
- **GET /product/:id**
 - Detalhe do produto
- **GET /search?query=&cat=&page=&limit=**
 - Lista de produtos com base na pesquisa e filtros realizados (página/filtro simples)
 - query = pesquisa a ser realizada | cat = categoria a ser pesquisada
 - page = número da página | limit = quantidade de produtos da página
- **POST /cart**
 - Recebe os itens do carrinho e retorna o resumo atualizado (subtotal, frete, desconto e total).
 - body: [{ items:[{productId,qty}], cupomCode }]
 - response: [{ items:[{productId,name,price,qty,image}], subtotal, freight, discount, total }]

Regras/observações

- Retornar **status codes** adequados: 200, 404, 500.
- Validar parâmetros de busca (ex.: search, page, limit ≥ 0).
- **Banco de dados**: Integração e persistência em um banco relacional.
- **Documentação mínima** (Swagger/OpenAPI) com rotas e exemplos de resposta.
- **Frete simulado**: freight = subtotal ≥ 200 ? 0 : 25 (padrão sugerido).
- **Cupom**: aceitar código simples (ex.: URI10 $\rightarrow$ 10% de desconto).

4) Critérios de Aceite

- Projeto funcional e conter **front APIs** com fluxo funcional: Home $\rightarrow$ Busca $\rightarrow$ Detalhes $\rightarrow$ Carrinho.
- Respostas **JSON** padronizadas; **status codes** corretos; erros tratados.
- **README** com detalhamento das tecnologias e libs utilizadas, como rodar, portas, rotas principais e exemplos de requisição.
- **Entrega na data estipulada em sala através do GitHub Classroom:**



<https://classroom.github.com/a/kMMwWf6D>

5) Requisitos bônus extra (+0,2pts)

Requisito opcional a fim de demonstrar domínio técnico avançado, implementar recursos que elevem a qualidade do projeto, com possibilidade de **bônus de até +0,2 pontos** na nota final. As melhorias devem ser **descritas e justificadas no README com evidências práticas**. Itens que podem ser considerados:

- **POST /login**
 - Realizar login
- **POST /register**
 - Cadastro do cliente, com dados mínimos (Nome, E-mail, Senha Criptografada)
- **GET /cart**
 - Carrinho de compras persistido na conta do usuário
 - response: [{ items:[{productId,name,price,qty,image}], subtotal, freight, discount, total }]
- **PUT /cart/items**
 - body: { productId, qty } (adicionar/atualizar item)
- **PUT /cart/items/:productId $\rightarrow$**
 - body: { qty } (alterar quantidade)
- **DELETE /cart/items/:productId**
 - Remover item do carrinho do usuário

- Criar uma página para administração dos produtos no Front-end (**/admin**), reutilizando as APIs já criadas para criação e edição dos produtos, garantindo segurança e permitindo apenas que usuários administrativos façam a adição.
 - Usuários administrativos serão definidos com uma flag em seu perfil **isAdmin=true**
- **Regras/observações**
 - Estrutura de código organizada em camadas com middlewares
 - **Tratamento** de erros 401 404 e 500.
 - **Documentação mínima** (Swagger/OpenAPI) com rotas e exemplos de resposta.

Wireframe Sugerido

